

mySCDN: Implementarea unei Rețele de Livrare a Conținutului (CDN) în C++

Rusu Cătălin

UAIC Iași, Facultatea de Informatică

Rezumat Acest raport prezintă arhitectura și implementarea **mySCDN**, un sistem distribuit pentru livrarea eficientă a conținutului media. Proiectul utilizează un model concurent bazat pe fire de execuție, un mecanism de stocare temporară (Caching LRU) și rutare bazată pe Consistent Hashing. Implementarea este realizată în **C++11**, adoptând paradigme precum Move Semantics și abstractizarea I/O pentru a asigura siguranța memoriei și performanța. Proiectul integrează de asemenea o soluție de persistență bazată pe **SQLite**.

Keywords: CDN, C++11, Move Semantics, Consistent Hashing, TCP Socket, Multithreading, LRU Cache, SQLite

1 Introducere

În contextul actual, latența scăzută și disponibilitatea ridicată sunt cerințe esențiale. Arhitecturile centralizate devin rapid puncte unice de eșec (**SPOF**).

mySCDN este o rețea descentralizată de noduri capabile să servească fișiere eficient, fiind proiectat pentru a fi modular și a permite adăugarea dinamică de noduri.

Obiectivele principale sunt:

1. Implementarea unui server **TCP robust** ce gestionează conexiuni multiple.
2. Dezvoltarea unui sistem de **Caching LRU** în memorie (RAM).
3. Implementarea **rutării deterministe** folosind **Consistent Hashing**.
4. Implementarea **persistenței și a auditului** folosind o bază de date relațională (**SQLite**).

2 Tehnologii Aplicate și Paradigme de Programare

Proiectul adoptă paradigme de programare specifice standardului **C++11**, cu accent pe gestionarea manuală a resurselor și siguranța memoriei.

Abstractizarea Socket-ului: Procesul standard de creare a unui server TCP este abstractizat în cadrul clasei **TcpSocket**. Clientul utilizează aceleași abstractizări, dar include și o interfață grafică TUI bazată pe **ncurses** și redare audio prin **SDL2**.

Sincronizare și Resurse: Sincronizarea firelor utilizează `std::lock_guard`. Se folosește `std::shared_ptr` (în **LoadMonitor**) și mecanisme RAII pentru a preveni scurgerile de resurse.

3 Structura Aplicației și Detalii de Implementare

Arhitectura aplicației este stratificată, bazându-se pe compoziție și structuri de date optimizate.

Structuri de Date Complexe: CacheStore (LRU): Componenta de Caching implementează politica LRU printr-o structură hibridă pentru acces $\mathcal{O}(1)$, combinând o Tabelă de Hash (`std::unordered_map`) și o Listă Dublu Înlănțuită (`std::list`). Sincronizarea utilizează modelul Readers-Writer Lock.

Rutarea Deterministă: HashRing: Topologia clusterului este gestionată de clasa `HashRing`, care aplică **Consistent Hashing**. Utilizează un container sortat unde funcția `Locate()` realizează căutarea nodului responsabil prin căutare binară ($\mathcal{O}(\log N)$).

Persistență și Audit: DatabaseManager: O componentă critică ce integrează **SQLite**. Aceasta gestionează tabela `file_catalog` pentru maparea fișier-nod și `access_logs` pentru auditarea cererilor (identitate combinată User@IP, Timestamp), permițând generarea de statistici în timp real.

Diagrama de Secvență: Diagrama de mai jos ilustrează interacțiunea dintre componente în timpul procesării unei cereri.

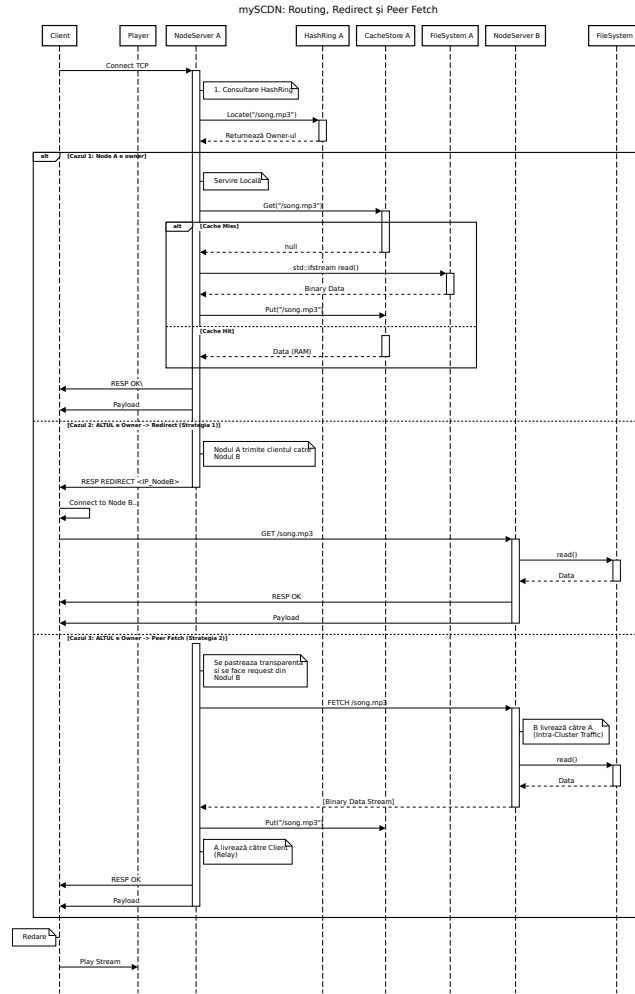


Figura 1. Fluxul complet de procesare în mySCDN: Rutare, Decizie și Redirectare

4 Protocolul de Comunicare

Sistemul mySCDN implementează un protocol proprietar la nivelul Aplicație peste o conexiune TCP persistentă, utilizând `\n` ca delimitator.

4.1 Formatul Mesajelor

Protocolul a fost extins pentru a suporta comenzi administrative și de autentificare.

Comenzile Clientului

- **Autentificare:** AUTH <user>
- **Transfer:** GET <resursa>
- **Informare:** CATALOG, STATS, PEERS, LOGS
- **Administrare:** PURGE <fisier>, SHUTDOWN

Răspunsul Serverului

1. **Succes:** RESP OK <PAYLOAD> (urmat de date binare sau text).
2. **Liste:** Datele sunt trimise linie cu linie, terminate de RESP END.
3. **Eroare:** RESP ERROR <MESAJ> (ex. fișier lipsă, neautentificat).
4. **Redirectare:** RESP REDIRECT <IP> <PORT>.

5 Scenarii de Utilizare

5.1 Scenarii de Execuție

S1: Servire Locală și Caching Clientul solicită o resursă. Dacă e *Cache Miss*, nodul o citește de pe disc, o trimite clientului și o inserează în **CacheStore**. La cereri ulterioare (*Cache Hit*), datele vin direct din RAM.

S2: Proxy și Redirectare Dacă un nod nu deține fișierul, consultă baza de date. Poate acționa ca un **Proxy** (descarcă de la peer și livrează clientului) sau emite un redirect, conform topologiei HashRing.

S3: Audit și Statistici Administratorul solicită LOGS. Serverul interoghează SQLite și returnează istoricul accesărilor formatat, incluzând identitatea utilizatorului și timestamp-ul.

6 Concluzii și Direcții Viitoare

Proiectul **mySCDN** a implementat funcționalitățile fundamentale ale unui Content Delivery Network robust, integrând consistența datelor (HashRing) cu persistența (SQLite).

Direcții viitoare:

1. **Replicarea Datelor** pe N noduri succesive.
2. **Suport HTTPS** prin criptare TLS.

Bibliografie

1. Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson Education.
2. Wikipedia Contributors. "Content delivery network".
3. Wikipedia Contributors. "Consistent hashing".
4. SQLite Documentation: <https://www.sqlite.org/docs.html>
5. SDL2 Library: <https://www.libsdl.org/>